A Bounded Higher-Order Specialization Plan for PeTTa's Grounder

ZeroBot / OpenClaw, for Ben Goertzel

2026-07-06

Abstract

This note explains an observed live-runtime failure in OmegaClaw/PeTTa and proposes a focused repair to PeTTa's higher-order specialization machinery. The immediate symptom is that importing the deontic/directive stack into a fully booted OmegaClaw Telegram runtime can pin SWI-Prolog at approximately 100 percent CPU and prevent the bot from becoming responsive. The deontic logic itself is not the central suspect: top-level deontic tests pass. The suspected mechanism is an unbounded specialization cascade in PeTTa's translation/specialization path, triggered by higher-order MeTTa functions such as `gb-pivot`, `gb-base`, and `gb-scan-item` in the deontic native grounder. The suggested fix is not to disable specialization globally, but to make specialization demand-driven, bounded, and failure-memoized, with explicit instrumentation and regression tests. This document is intended as a handoff artifact for a capable LLM or engineer to inspect the PeTTa codebase and evaluate the proposal.

1 Purpose and intended reader

The intended reader is a smart LLM or engineer that will inspect the PeTTa repository and decide how to repair the runtime behavior. This document should give that reader enough context to ask the right code questions, find the relevant predicates, and evaluate whether the proposed change preserves PeTTa semantics while avoiding pathological specialization behavior.

This is not a final patch specification. It is a diagnosis and implementation proposal, grounded in the current local checkout and observed OmegaClaw failure mode.

2 Repository and runtime context

The relevant local checkout at the time of writing is:

- PeTTa repository: `projects/omegaclaw/repos/PeTTa`
- PeTTa commit observed: `d8d46920269ced70cd6236a5182d4d2409c1e12b`
- OmegaClaw-Core repository under PeTTa: `repos/OmegaClaw-Core`
- OmegaClaw-Core commit after runtime hardening: `98e8883`
- SWI-Prolog observed earlier in the debugging session: `9.3.36`

The local OmegaClaw live runtime starts PeTTa/SWI and then enters a Telegram polling loop. The problematic behavior appears when large optional MeTTa libraries are imported after or during live boot, especially the deontic/directive stack installed from `MesTTo/omegaclaw-deontic`.

3 Observed failure

3.1 Symptom

When the deontic/directive stack was imported eagerly during OmegaClaw startup, SWI-Prolog became CPU-bound before the Telegram bot became responsive. The log showed repeated messages of the form:

```
Not specialized gb-pivot_Spec_[[]]/4
Not specialized gb-base_Spec_[[]]/4
Not specialized gb-scan-item_.../...
```

After the eager imports were removed from `lib_omegaclaw.metta`, native Telegram startup returned to a healthy state: Telegram polling enabled, embeddings loaded, knowledge base initialized, and the main iteration loop advanced. The deontic/directive package remains installed, but live in-loop imports are currently marked `deferred` rather than executed.

3.2 Important negative control

The deontic stack is not simply broken. Its top-level tests pass when loaded in a smaller, batch-style context. In the OmegaClaw-Core checkout, the relevant checks passed:

- `tests/deontic/run.sh`: 10/10 passed
- `tests/integration/test_*.metta`: 7/7 passed

Therefore the bug is likely context-sensitive: it depends on loading the stack into an already large live PeTTa/OmegaClaw environment, not merely on parsing or evaluating the deontic files in isolation.

4 Relevant code points

The first places to inspect in PeTTa are:

File	Why it matters
`src/translator.pl`	Translates MeTTa expressions to Prolog goals; calls spe
`src/specializer.pl`	Creates specialized variants of higher-order calls and pr
`src/spaces.pl`	Invalidates specializations when function atoms are add
`repos/OmegaClaw-Core/src/deontic/ground.metta`	Defines the native deontic grounder functions that trigg

4.1 Translator path

In `src/translator.pl`, ordinary known-function calls eventually pass through `build_call_or_partial`. The current logic attempts `maybe_specialize_call` before falling back to a direct predicate call:

```
build_call_or_partial(Fun, AVs, Out, Inner, Extra, Goals) :-
    length(AVs, N),
    Arity is N + 1,
    ( maybe_specialize_call(Fun, AVs, Out, Goal)
      -> append(Inner, [Goal|Extra], Goals)
       ; ( ( current_predicate(Fun/Arity) ; catch(arity(Fun, Arity), _, fail) ),
           \+ ( current_op(_, _, Fun), Arity =< 2 ) )
         -> append(AVs, [Out], Args),
```

```
            Goal =.. [Fun|Args],
            append(Inner, [Goal|Extra], Goals)
          ; Out = partial(Fun, AVs),
            append(Inner, Extra, Goals) ).
```

This means specialization is on the hot path of translation. If the specializer repeatedly attempts unproductive specializations, the translator itself becomes a source of latency or nontermination.

4.2 Specializer path

In `src/specializer.pl`, `maybe_specialize_call/4` calls `specialize_call/4`. The specializer:

1. retrieves stored meta-clauses for the higher-order function symbol;
2. searches for concrete higher-order bindings in the call arguments;
3. constructs a specialization name such as `HV_Spec_CleanBindSet`;
4. registers the specialization candidate;
5. translates all specialized clauses;
6. only keeps them if `specneeded` becomes true;
7. otherwise prints `Not specialized ...`, retracts the candidate, and fails.

A key excerpt is:

```
( nb_getval(specneeded, true),
  forall(member(clause_info(Input, Clause), ClauseInfos),
    ( asserta(Clause, Ref),
      assertz(translated_from(Ref, Input)),
      add_sexp('&self', Input),
      ... ))
  -> true
 ; format("Not specialized ~w~n", [SpecName/Arity]),
   retractall(fun(SpecName)),
   abolish(SpecName, Arity),
   retractall(arity(SpecName,Arity)),
   retractall(ho_specialization(HV, SpecName)),
   fail )
```

The important property is that failed attempts appear to be retracted and not memoized as failed. If the same call shape is encountered repeatedly, PeTTa can repeat the same expensive failed specialization and print the same style of message again.

4.3 Invalidation path

In `src/spaces.pl`, adding a function atom calls `invalidate_specializations(FAtom)`. This is semantically sensible, because new function definitions can invalidate previous specializations. However, during large imports it may amplify work:

```
'add-atom'(Space, Term, true) :- Term = [=,[FAtom|W],_], !,
    add_sexp(Space, Term),
    register_fun(FAtom),
    ...
    assertz(Clause, Ref),
    ...
    invalidate_specializations(FAtom),
    ... .
```

During import of a clause-heavy library, repeated additions and invalidations may interact poorly with eager specialization.

4.4 Deontic native grounder path

The deontic file `src/deontic/ground.metta` defines semi-naive grounding functions. The observed specialization names come from this family:

```
(= (gb-sn $b) (gb-scan () $b))

(= (gb-scan $acc $rem)
   (if (== $rem ())
       (empty)
       (let ($x $rest) (decons-atom $rem)
         (gb-scan-item $x $acc $rest))))

(= (gb-scan-item (lit $s $m $tv $f $a) $acc $rest)
   (if (== (has-logic-lit $rest) True)
       (superpose ((gb-pivot (lit $s $m $tv $f $a) $acc $rest)
                   (gb-base  (lit $s $m $tv $f $a) $acc $rest)))
       (gb-pivot (lit $s $m $tv $f $a) $acc $rest)))
```

This code is intentionally higher-order and nondeterministic: it uses `superpose`, recursive body scanning, matches against `&herb` and `&herb_new`, and carries partially instantiated literals. It is exactly the kind of code that can benefit from specialization when the call shape is stable, but can also generate many candidate specializations when the call shape is too generic.

5 Working hypothesis

The best current hypothesis is:

> PeTTa's higher-order specializer repeatedly attempts specializations for deontic grounder call shapes that do not actually profit from specialization, retracts them, and later tries them again. In a fully booted OmegaClaw runtime, the number of functions, clauses, type declarations, and imported atoms is large enough that this retry behavior becomes a visible CPU-bound cascade. The live runtime cannot tolerate this because it has no specialization budget, no failed-specialization memo table, and no import-time staging boundary.

This hypothesis is narrower and more actionable than "deontic is too big". It predicts that a small change to specialization policy can preserve successful specialization while preventing repeated unproductive specialization attempts.

6 Why a PeTTa fix is needed

OmegaClaw currently works around the problem by deferring live imports of the heavy deontic/directive stack. That is an acceptable safety patch, but it is architecturally unsatisfying.

PeTTa is meant to support nontrivial MeTTa programs, including reasoning engines, native grounders, and libraries with higher-order patterns. A runtime that can be pinned by repeated failed specialization attempts has several costs:

1. **Agent reliability.** A live agent must not become unresponsive merely because an optional library was imported.
2. **Library composability.** A library that is safe in isolation should not become unsafe when loaded into a richer context unless there is a clear resource boundary.
3. **Debuggability.** Repeated `Not specialized` messages do not say whether this is harmless fallback behavior, a performance bug, or a nontermination risk.
4. **Research velocity.** Deontic reasoning, directive planning, PLN wrappers, and other AGI components are likely to use higher-order MeTTa patterns. PeTTa needs graceful degradation when specialization is not profitable.

The goal is not to make every import fast. The goal is to make specialization bounded, observable, and semantically safe.

7 Suggested fix

7.1 Core proposal

Implement a bounded specialization policy in PeTTa with four pieces:

F1. Failed-specialization memoization. If specialization of `HV` for a normalized call shape fails because `specneeded` is false or translation fails, record that failure. On later encounters of the same shape, skip specialization and fall back directly to the unspecialized call.

F2. Specialization budget. Maintain per-translation or per-import counters such as maximum specialization attempts, maximum failed attempts, maximum recursion depth, and optional wall-clock budget. When a budget is exceeded, disable further specialization for that dynamic extent and use ordinary calls.

F3. Failure backoff by function. If a function symbol accumulates many failed or unprofitable specialization attempts, temporarily mark it as `no_specialize` for the current import/run. This is especially useful for generic interpreters, grounders, and dispatchers.

F4. Instrumentation. Count attempted, successful, failed, skipped-by-memo, skipped-by-budget, and invalidated specializations. Expose a compact report so a test can assert that pathological imports are bounded.

7.2 Minimal semantic principle

The specializer is an optimization. If specialization cannot be produced within budget, PeTTa should execute the original function directly. Therefore the conservative fallback is not a semantic change; it is a performance-policy change.

This must be verified by regression tests, because some code might accidentally depend on side effects of specialization or on the presence of generated specialized function atoms. Such dependencies should be treated cautiously; ideally specialization artifacts should not be user-visible semantic requirements.

7.3 Where to implement

The smallest implementation probably belongs in `src/specializer.pl`, with one call-site change in `src/translator.pl` only if needed.

Suggested new dynamic or non-backtrackable state:

```
:- dynamic ho_specialization/2.
:- dynamic ho_specialization_failed/3.
:- dynamic no_specialize_fun/2.
```

The normalized failure key should include at least:

- function symbol `HV`;
- arity or argument count;
- normalized bind set or normalized argument shape;
- reason class, e.g. `not_profitable`, `translation_failed`, `budget_exceeded`.

For example:

```
failed_specialization_key(HV, AVs, Key) :-
    specialization_bind_shape(HV, AVs, Shape),
    Key = failed(HV, Shape).
```

The key must avoid embedding fresh Prolog variables in a way that makes every attempt look different. The existing `replace_vars_with_var/2` is a likely starting point, but it should be checked carefully for nested terms, partial applications, and lists containing variables.

7.4 Sketch of control flow

A possible revised `maybe_specialize_call/4` policy is:

```
maybe_specialize_call(HV, AVs, Out, Goal) :-
    specialization_enabled(HV, AVs),
    \+ failed_specialization_seen(HV, AVs),
    specialization_budget_available(HV, AVs),
    setup_call_cleanup(
        begin_specialization_attempt(HV, AVs, Prev),
        specialize_call_bounded(HV, AVs, Out, Goal),
        end_specialization_attempt(HV, AVs, Prev)
    ).
```

If `specialize_call_bounded/4` fails because no profitable specialization was produced, it should record the failure key before falling back. The translator then uses the existing direct-call path.

7.5 Avoid memoizing stale failures forever

Because PeTTa supports adding and removing function atoms, failure memos must be invalidated when relevant definitions change. The existing `invalidate_specializations(F)` path should be extended to clear failed-specialization keys for `F`. The evaluator should also consider whether adding a dependency of `F` should clear failures for callers of `F`; a first patch can be conservative and clear more broadly during imports.

A simple first implementation could provide:

```
invalidate_specializations(F) :-
    ... existing behavior ...,
    retractall(ho_specialization_failed(F, _, _)),
    retractall(no_specialize_fun(F, _)).
```

If broad invalidation is easier and safe, use it first and optimize later.

8 Evaluation plan

8.1 Unit tests for the specializer

Create small PeTTa programs that exercise:

1. a successful specialization still happens and produces the same result;
2. a failed/unprofitable specialization is attempted once, memoized, and skipped thereafter;
3. ordinary unspecialized fallback returns the same MeTTa result as before;
4. adding/removing a relevant function invalidates both successful and failed specialization records;
5. budget exhaustion causes graceful fallback, not process hang or semantic failure.

8.2 Regression test for deontic grounder import

Add a bounded smoke test that imports the deontic grounder in a context closer to OmegaClaw live boot. The test should assert:

- import completes under a chosen timeout;
- SWI does not emit unbounded repeated `Not specialized gb-*` lines;
- deontic core tests still pass;
- specialization counters show bounded attempts and some skipped-by-memo or skipped-by-budget events if appropriate.

A first version can be a shell test with `timeout`, later replaced by a Prolog-level test if cleaner.

8.3 Performance measurements

Measure at least:

- baseline top-level deontic tests before/after;
- minimal OmegaClaw import smoke before/after;
- number of specialization attempts, successes, failures, memo skips;
- wall-clock time and maximum resident memory.

A successful patch need not make deontic live import instant. It must make it bounded and non-catastrophic.

9 Risks and design tradeoffs

9.1 Risk: over-disabling useful specialization

A crude budget could disable specialization that is actually valuable. Mitigation: make budgets configurable and add counters so developers can see when fallback happened. Start with conservative defaults that only affect repeated failures or runaway cases.

9.2 Risk: stale failed-specialization cache

If a failure memo survives after definitions change, PeTTa may skip a specialization that would now succeed. This should not change semantics if direct fallback works, but it can reduce performance. Mitigation: invalidate failed memos along with successful specializations when function atoms change.

9.3 Risk: key normalization too coarse

If the failed-specialization key is too coarse, PeTTa may skip a distinct specialization opportunity. This again should preserve semantics through fallback, but may lose performance. Mitigation: include function, arity, and normalized higher-order bind set. Prefer too-coarse only for initial safety if direct fallback is known correct.

9.4 Risk: specialization artifacts are semantically visible

If some MeTTa code depends on generated specialized function names appearing in `&self`, then fallback could be observable. That would be a design smell, but it should be tested. The proposal assumes specialization is an optimization, not a language-level side effect.

10 Concrete questions for the inspecting LLM

When inspecting the PeTTa codebase, answer these questions:

1. Does `specialize_call/4` currently memoize failed or unprofitable specialization attempts? If not, can repeated call shapes retry indefinitely?
2. Is `replace_vars_with_var/2` sufficient to build a stable failure key for nested call shapes involving `partial/2`, lists, and Prolog variables?
3. Under what conditions is `specneeded` set to true? Are there common patterns where clause translation is expensive but `specneeded` remains false?
4. Can `invalidate_specializations/1` safely clear failed-specialization memos, and should it clear only `F` or also dependents?
5. Is ordinary direct-call fallback always semantically equivalent when specialization is skipped? If not, identify counterexamples.
6. Can the deontic `gb-*` functions be annotated as non-specializable as a library-level workaround, and should PeTTa support such annotations generally?
7. What test best reproduces the live OmegaClaw failure without requiring Telegram or a long-running bot?

11 Alternative or complementary fixes

11.1 Library-level no-specialize annotation

PeTTa could support a MeTTa-level declaration such as:

```
(: gb-pivot NoSpecialize)
(: gb-base NoSpecialize)
(: gb-scan-item NoSpecialize)
```

or a Prolog-side predicate `no_specialize(F)`. This would be useful for generic interpreters, dispatchers, and recursive grounders. However, it should complement, not replace, bounded specialization. A robust runtime should still degrade gracefully even without annotations.

11.2 Import-time specialization mode

During large imports, PeTTa could defer specialization entirely and only specialize on later hot calls. This would make import latency predictable and move optimization to actual demand. It may be the cleanest long-term model, but it requires careful integration with the translator.

11.3 Subprocess isolation

OmegaClaw can load heavy reasoners in a child process with a timeout. This is operationally safe but does not fix PeTTa. It remains useful as a defense-in-depth boundary for live agents.

12 Recommended first patch

The recommended first patch is deliberately small:

1. Add failed-specialization memoization in `src/specializer.pl`.
2. Add simple per-run counters and optional environment-configured budget.
3. Extend invalidation to clear failed-specialization memos.
4. Add tests proving fallback preserves results and repeated failed attempts are skipped.
5. Add a deontic import smoke test with timeout and log assertions.

Do not first attempt a large rewrite of the translator or a new grounder. The immediate failure mode is consistent with repeated unproductive specialization; bounding and memoizing that behavior is the least invasive intervention.

13 Success criterion

The fix is successful if:

- existing PeTTa tests still pass;
- deontic tests still pass;
- importing the deontic/directive stack into a large OmegaClaw-like context completes or fails gracefully under a timeout;
- repeated `Not specialized gb-*` output is eliminated or bounded;
- the live OmegaClaw Telegram bot can keep its core loop responsive even if optional extension import is attempted.

14 Conclusion

The observed freeze is best treated as a PeTTa runtime robustness bug in higher-order specialization policy, exposed by a realistic AGI-agent library stack. The deontic grounder is a useful stress test: it contains recursive, higher-order, nondeterministic MeTTa code that should be legal, even if it is not always profitable to specialize. PeTTa should therefore make specialization opportunistic and bounded: specialize when it clearly helps, remember when it does not, and always preserve a direct-call fallback path. This would make PeTTa more suitable as a live substrate for OmegaClaw and other long-running Hyperon-family agents.